

BHTTP/1

HTTP semantics in a binary framing over TCP. Keywords MUST, MUST NOT, SHOULD, MAY are as in RFC 2119. All integers are big-endian.

1 Connection and Preface

One TCP connection carries many sequential requests (keep-alive). A client MUST NOT open a second connection to complete a request. Each endpoint sends an 8-byte preface before any frame. The client sends first; the server replies with min(client version, highest version it supports). This document defines version 1; version 0 is invalid. Reserved bytes: senders MUST send 0, receivers MUST ignore. A server whose peer's magic is not BHTTP MUST close without sending anything. A client receiving an unsupported version MUST close.

```
+-----+-----+-----+
| "BHTTP" (4) | Ver (u8) | Reserved (3) | 42 48 54 50 01 00 00 00
+-----+-----+-----+
```

2 Frame Header (8 bytes, fixed)

```
+-----+-----+-----+-----+
| Length (16) | Type(8) | Flg(8) | R | Stream ID (31) |
+-----+-----+-----+-----+
```

Field	Rule
Length	Payload bytes after the header. A sender MUST NOT send more than 16384 (MAX_PAYLOAD). A receiver that reads a larger Length MUST treat it as a connection error and MUST NOT read or allocate the payload.
Type	A receiver that does not recognise Type MUST read and discard Length bytes and continue.
Flags	Type-specific. Senders MUST set undefined bits to 0; receivers MUST ignore them. Bit 0x01 is END_STREAM (REQUEST, RESPONSE, DATA): the last frame the sender sends on that stream. [B5]
R	Reserved. Senders MUST send 0. Receivers MUST ignore it and use only the low 31 bits as Stream ID. [C3]
Stream ID	Client requests use odd IDs, strictly increasing, starting at 1. ID 0 is for connection-level frames. Even IDs are reserved for server-initiated streams (unused in v1). A client MUST NOT reuse or wrap IDs; before $2^{31}-1$ it MUST close and open a new connection.

3 Frame Types

Type	Name	Dir	Stream	Payload
0x01	REQUEST	C to S	odd	method u8, pathlen u16, path, header block
0x02	RESPONSE	S to C	= request	status u16, header block
0x03	DATA	S to C	= request	body bytes
0x04	ERROR	both	0	code u16, optional UTF-8 text

REQUEST. Method MUST be 0x01 (GET). Path MUST be non-empty and begin with "/". A v1 REQUEST MUST set END_STREAM. A client MUST NOT send a new REQUEST until the previous response has ended (one request in flight in v1). **RESPONSE.** Status MUST be 100..599. If END_STREAM is set the body is empty; otherwise (flags = 0) DATA frames follow on the same stream and the last has END_STREAM. [B5] REQUEST and RESPONSE MUST fit in one frame (no continuation); a sender whose payload would exceed 16384 MUST fail locally. **ERROR codes:** 1 PROTOCOL, 2 FRAME_SIZE, 3 TIMEOUT, 4 INTERNAL; unknown codes MUST be treated as PROTOCOL. A server MUST discard DATA from a client. Receivers MUST ignore frames on streams they did not open. [C5] A RESPONSE sent to a server, or a REQUEST sent to a client, is a connection error.

4 Header Blocks

A header block is zero or more fields packed to the end of the payload. Order carries no meaning except among duplicates of one name.

[B4]

```
Indexed name: 1xxxxxxx | vlen u8 | value          xxxxxxx = static index (1-based)
Literal name: 00000000 | nlen u8 | name | vlen u8 | value
```

1	2	3	4	5
host	user-agent	accept	if-none-match	content-type
6	7	8	9	10
content-length	etag	cache-control	last-modified	server

The static table is **1-based**; index 0 and indices above 10 are invalid. [C1] Names are ASCII; senders MUST lowercase, receivers MUST lowercase literal names. A literal name MUST NOT be empty. Name and value are at most 255 bytes. Values are opaque bytes; receivers MUST NOT assume a format beyond those in section 6. [B2] Senders SHOULD use the indexed form when the name is in the table; receivers MUST accept the literal form of any name. First bytes 0x01..0x7F are reserved and cannot be skipped, so a peer MUST NOT send them in v1.

5 Errors

Stream error: a fault in a frame whose boundaries were intact (malformed REQUEST payload, or a REQUEST without END_STREAM). The server MUST answer RESPONSE 400 on that stream and MUST keep the connection open.

Connection error: any fault after which the next frame boundary cannot be trusted, or a protocol violation: Length > 16384; EOF or timeout inside a frame; REQUEST with an even, zero or non-increasing Stream ID; a misdirected frame (section 3); DATA on stream 0; ERROR on a non-zero stream. The detector SHOULD send ERROR on stream 0 and MUST close, first half-closing its write side and draining input for a short bounded time so that unread input does not cause a TCP reset that destroys the ERROR. A peer receiving ERROR MUST close. Unknown frame types are neither kind of error.

6 Static File Serving (bserve)

The path is opaque bytes: no percent-decoding, no query string. It maps to a regular file beneath a configured root. A server MUST NOT serve a file outside the root, including through symbolic links. A path that is empty after "/", contains ".", ".." or empty segments, ends in "/", contains NUL, does not exist, is not a regular file, or resolves outside the root MUST yield 404 with an empty body and no distinguishing detail.

A 200 response carries content-type, content-length, etag, last-modified, cache-control and server. content-length is REQUIRED whenever DATA follows, and the DATA frames MUST total exactly that many bytes. [c4] etag is "<mtime hex>-<size hex>", quotes included. last-modified is IMF-fixdate in GMT, e.g. Thu, 08 Oct 2026 06:32:38 GMT. [B1] A request whose if-none-match lists the etag (or "") receives 304 with END_STREAM, no content-length and no body. content-type values are opaque to the protocol. [B2]

A server that cannot finish a body it started (the file shrank) MUST send ERROR(INTERNAL) and close. A client that sees EOF or ERROR before END_STREAM, or a body length different from content-length, MUST treat the response as failed even though a 200 was received.

7 Resource Limits

A receiver MUST NOT allocate for a payload before validating Length. Memory per connection MUST NOT exceed one maximum frame and SHOULD be zero when idle. A server SHOULD bound handshake, idle (waiting for a header), frame (completing a frame after its header) and write (each frame) time, sending ERROR(TIMEOUT) where a write is still possible, then closing. It SHOULD bound concurrent connections by ceasing to accept, so excess peers queue in the kernel backlog. Time bounds limit how long a hostile peer holds a slot, not how many it holds; per-address limits are out of scope for v1.

8 Design Decisions

Measured numbers are in docs/bench.md.

Decision	Reason	Rejected
Fixed 8-byte header	One aligned read; Length at offset 0 makes unknown types skippable	Variable header: breaks resync
16-bit Length, 16 KiB cap	4x less buffer memory per in-flight frame than 64 KiB; 4.8 KiB per idle connection. Measured cost: 671 vs 1044 MiB/s on loopback (per-frame syscalls)	24-bit (H2): unusable range. 64 KiB: memory, head-of-line delay
31-bit Stream ID	12.4 days at 1000 req/s; odd/even avoids negotiation	16-bit: 33 s
Version in preface	Zero per-frame cost; fails fast	Per-frame version
Static table + literal	Stateless: a bad frame cannot corrupt later ones	Dynamic table, Huffman
Stream vs connection error	Split by: can the next boundary be found?	One error class
404 for all path failures	Reveals nothing about files outside the root	403
os.Root path mapping	Containment inside open(); no check-then-open race	Clean + EvalSymlinks + prefix
Late payload buffer, no bufio	Idle connections hold no buffer	Per-connection 16 KiB or 4 KiB bufio

9 Version 2 Wishlist

Multiplexing (stream IDs exist), RST_STREAM, GOAWAY, flow control, a larger negotiated MAX_PAYLOAD, dynamic table and indexed name+value pairs (reserved first bytes), Range, varint lengths for long values, per-address limits, server-initiated streams.

A Worked Example: GET /index.html, Host localhost:9000

```
00 1e 01 01 00 00 00 01  header: len=30 REQUEST END_STREAM stream=1
01                          method GET
00 0b 2f 69 6e 64 65 78  pathlen=11, "/index"
2e 68 74 6d 6c            ".html"
81 0e 6c 6f 63 61 6c 68  host (idx 1), vlen=14, "localh
6f 73 74 3a 39 30 30 30  "ost:9000"
```

38 bytes against 50 for HTTP/1.1. Full annotated exchange: docs/hexdump.md. Tags such as [B5] mark spec fixes found while annotating (B) and interop testing (C).